

Considerare il seguente problema combinatorio, noto come Edge Colouring.

Dato in input un grafo non diretto $G = (V, E)$, si vogliono colorare gli archi del grafo con 3 colori (denotati con '1', '2' e '3') di modo che non esistano triangoli composti da archi tutti dello stesso colore.

1.1 Modellazione

Formalizzare il problema di trovare, dato un qualunque grafo $G = (V, E)$, una tale colorazione dei suoi archi, come una specifica di CSP. Preferire una notazione matematica di alto livello.

1.2 Istanziamento

Istanziare la specifica di cui alla sec:modelling per il grafo ottenuto da quello fig:searchspace ignorando le etichette e le direzioni degli archi (ovvero fondendo eventuali coppie di archi $X \rightarrow Y$ e $Y \rightarrow X$ in un unico arco), producendo quindi un CSP (X, D, C) .

1.3 Codifica in Minizinc

Scrivere la specifica di cui alla sec:modelling nel linguaggio Minizinc (<http://www.minizinc.org>).

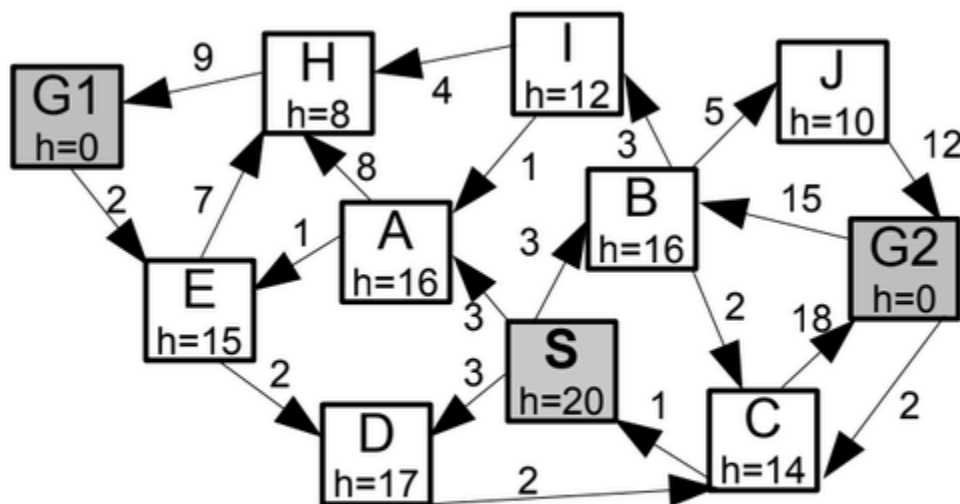


Figura 1.1: Un grafo

1.1 Modellazione

Per ogni arco $e \in E$, definiamo una variabile X_e = colore assegnato all'arco

Costruiamo il CSP.

Variabili

$X = \{X_e \mid e \in E\}$ Quindi abbiamo una variabile per ogni arco. Se per esempio:

Domini

Ogni arco può avere uno dei tre colori $D(X_e) = \{1, 2, 3\}$ per ogni $e \in E$.

Vincoli

Il problema dice “non devono esistere triangoli composti da archi tutti dello stesso colore”, prendiamo quindi un triangolo $T = \{e1, e2, e3\}$. I tre archi del triangolo non possono avere tutti lo stesso colore, ossia $!(X_{e1} = X_{e2} = X_{e3})$.

Questo deve valere per ogni triangolo del grafo. Se indichiamo con $TSet$ l'insieme dei triangoli di G , possiamo scrivere $\forall T = \{e1, e2, e3\} \in TSet(G) : !(X_{e1} = X_{e2} = X_{e3})$.

Quindi il nostro CSP(X, D, C) è composto da:

- $X = \{X_e \mid e \in E\}$
- $D(X_e) = \{1, 2, 3\}$
- $C = \{\forall T = \{e1, e2, e3\} \in TSet(G) : !(X_{e1} = X_{e2} = X_{e3})\}$

1.2 Istanziamento

Dalla figura otteniamo i vertici $V = \{G1, H, I, J, E, A, B, G2, D, S, C\}$. Gli archi, dopo aver eliminato direzioni ed etichette, sono:

1. G1-H
2. G1-E
3. H-E
4. H-I
5. H-A
6. A-E
7. A-I
8. I-B
9. A-S
10. S-B
11. S-D
12. S-C
13. E-D
14. D-C
15. B-C

16. B-J
17. B-G2
18. J-G2
19. C-G2

Quindi abbiamo $|E| = 19$ e pertanto avremo 19 variabili CSP, una per ogni arco. Assegnamo alla variabile X_i la coppia seguendo il precedente elenco, ad esempio X_{10} è la coppia S-B.

Quindi $X = \{X_1, \dots, X_{19}\}$ e $D(X_i) = \{1, 2, 3\}$ per ogni i .

Dobbiamo ora cercare tutti i triangoli nel grafo.

1. $T_1 = \{X_3, X_5, X_6\}$ per H-A-E
2. $T_2 = \{X_4, X_5, X_7\}$ per H-I-A
3. $T_3 = \{X_{10}, X_{12}, X_{15}\}$ per B-S-C
4. $T_4 = \{X_{15}, X_{17}, X_{19}\}$ per B-C-G2
5. $T_5 = \{X_{16}, X_{17}, X_{18}\}$ per B-J-G2
6. $T_6 = \{X_{11}, X_{14}, X_{12}\}$ per S-D-C
7. $T_7 = \{X_1, X_2, X_3\}$ per G1-H-E
8. $T_8 = \{X_3, X_5, X_6\}$ per H-A-E

Per ciascun triangolo imponiamo che non tutti e tre gli archi abbiano lo stesso colore. Quindi:

- $!(X_3 = X_5 = X_6)$
- $!(X_4 = X_5 = X_7)$
- ... così via per tutti gli 11 triangoli

Quindi il nostro $CSP(X, D, C)$ è composta da:

- $X = \{X_1, \dots, X_{19}\}$
- $D(X_i) = \{1, 2, 3\}$
- $C =$ i vincoli sopra

1.2 Codifica in Minizinc

```
int: m = 19;
int: ntri = 8;

array[1..m] of var 1..3: color;

array[1..ntri, 1..3] of int: triangles =
  array2d(1..ntri, 1..3,
    [
      3, 5, 6,      % H-A-E
      4, 5, 7,      % H-I-A
      10, 12, 15,    % B-S-C
```

```

        15, 17, 19,      % B-C-G2
        16, 17, 18,      % B-J-G2
        11, 12, 14,      % S-D-C
        1, 2, 3,         % G1-H-E
        3, 5, 6          % H-A-E

    ]);

constraint
    forall(t in 1..ntri) (
        not (
            color[triangles[t,1]] =
            color[triangles[t,2]] /\
            color[triangles[t,2]] =
            color[triangles[t,3]]
        )
    );

solve satisfy;

output [
    "color = ", show(color), "\n"
];

```